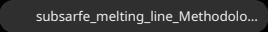


Let's start planning it. I will do all ml side work on jupyter notebook. but for downloading the data i want to use a Python script with heavy using of multiprocessing. Use suiteable frame work like dask or concurrent or normal multiprocessing. Which will have better system ram efficiency and also will have better retry policie. I want it to be seamless data download.

lets decide first which glaciears we are going to use. we will use clip to the that glaciear only for input data. We will also need data to validate and identify those.

Good. Before model training, the **data acquisition architecture** must be rock solid, otherwise everything downstream becomes pain. I went through your methodology and the glacier selection section.



1. First decision: Data download architecture

Since you're doing ML in Jupyter but want **heavy multiprocessing + seamless retry + RAM efficiency**, I would **not use Dask as the primary downloader**.

**Best choice:** `asyncio + ProcessPoolExecutor + aiohttp + bounded queue`

with `xarray/rasterio/zarr` for storage.

Why:

| Framework                              | RAM Efficiency     | Retry Policy        | Massive Downloads | Stability   | My Verdict |
|----------------------------------------|--------------------|---------------------|-------------------|-------------|------------|
| <code>multiprocessing</code>           | Medium             | Manual              | Good              | Stable      | OK         |
| <code>concurrent.futures</code>        | Good               | Manual but easy     | Very good         | Stable      | Better     |
| <code>Dask</code>                      | Heavy RAM overhead | Weak for flaky APIs | Overkill          | Complex     | No         |
| <code>Ray</code>                       | Heavy              | Good                | Excellent         | Overkill    | No         |
| <code>asyncio + executor hybrid</code> | Best               | Excellent           | Excellent         | Very stable | Use this   |

Why not Dask?

Dask shines for **distributed computation**, not downloading.

Problems for your case:

- Higher scheduler overhead
- More RAM usage

- Harder retry logic
- Satellite APIs fail randomly → retries matter more than compute

Your bottleneck will be:

**network + API rate limit + disk IO**, not CPU.

### Recommended architecture

```
Coordinator
|
Task Queue
|
Async Download Workers (50-200)
|
Retry Manager
|
Validation Checksum
|
GeoTIFF/Zarr Storage
```

Each file:

```
task -> download -> verify -> retry if corrupted -> clip AOI -> save
```

Retry policy:

```
Max retries: 8
Exponential backoff:
2s → 5s → 10s → 30s → 60s
```

Also:

### Important: never keep rasters in RAM

Store immediately to disk.

Use:

```
<> Python
zarr
```

instead of loading huge TIFFs repeatedly.

Folder structure:

```
raw_data/
  glacier_name/
    sentinel1/
    sentinel2/
    era5/
    dem/
   alos2/
    modis/

processed/
clipped/
training_ready/
```

## 2. Glacier selection — this matters a LOT

You cannot start with 12 glaciers immediately.

That will become a nightmare for:

- downloading
- alignment
- labeling
- validation
- debugging

We should go **Phase-wise**.

Your methodology says:

- Western Himalaya (4)
- Nepal Himalaya (3)
- Greenland (2)
- Antarctica (3)

subsarfe\_melting\_line\_Methodolo...

For research-quality work, I recommend:

---

## Phase 1 — Pilot glaciers (3 glaciers only)

Goal:

**make pipeline work + prove hypothesis**

Choose glaciers with:

- abundant data
- easy boundary shapefile
- GPR/AWS availability
- Sentinel coverage
- prior studies

### Glacier 1 (Primary Himalayan)

Khumbu Glacier

Why:

- massive literature
- ICIMOD support
- weather station data
- GPR available
- Sentinel coverage excellent

Most beginner-friendly Himalayan glacier.

---

### Glacier 2 (Ladakh / J&K)

Drang-Drung Glacier

Why:

- western Himalaya
  - strong seasonal melt
  - easier geometry
  - Sentinel-1 works well
  - important for your Himalayan focus
-

### Glacier 3 (Validation glacier)

Helheim Glacier

Why:

- publicly available cryosphere datasets
- Greenland has much better validation data
- easier benchmarking

This helps reviewers trust the model.

---

### Why NOT start with Siachen?

Siachen Glacier

Bad first glacier because:

- military restrictions
- poor field validation
- complex debris cover
- data inconsistencies
- very large AOI → expensive processing

Maybe later in Phase 3.

---

## 3. Glacier clipping strategy

You mentioned:

we will use clip to the glacier only

Correct approach.

We should download a **buffer AOI**, not exact glacier polygon.

Recommended:

glacier boundary + 2-5 km buffer



Why?

You need:

- surrounding terrain context
- thermal spillover
- topographic radiation
- watershed influence

After download:

```
download big tile
- reproject
- clip to glacier polygon
- create glacier mask
- save masked raster
```



We'll need:

**glacier inventory source**

Use:

Randolph Glacier Inventory

This should be the **single source of truth** for glacier polygons.

Don't manually draw polygons.

---

## 4. Validation datasets (critical)

You said:

  We will also need data to validate and identify those

Correct.

For each glacier we need:

### A. Glacier polygon

Source:

RGI

### B. GPR data (highest priority)

for melt-depth validation.

### C. AWS weather stations

for weak labels.

### D. Mass balance datasets

for indirect validation.

### E. Surface melt products

for sanity checking.

---

## Suggested validation stack

| Validation           | Purpose           | Priority |
|----------------------|-------------------|----------|
| GPR transects        | Ground truth      | Highest  |
| AWS temperature/snow | Weak labels       | High     |
| SNOWPACK simulation  | Pseudo labels     | High     |
| GRACE                | Mass balance      | Medium   |
| ICESat-2 altimetry   | Thickness changes | Medium   |

---

## My recommendation for start

Let's freeze Phase 1:

### Pilot glaciers

1. Khumbu
2. Drang-Drung

3. Helheim

Then:

Data sources first

- 1. Sentinel-1
- 2. Sentinel-2
- 3. ERA5
- 4. DEM
- 5. MODIS LST

Skip ALOS-2 initially.

Reason:

L-band data workflow is painful. Start with C-band and prove pipeline first.

Then later add ALOS-2 as an ablation study (your methodology already supports this idea).

Question before we proceed:

Do you want the thesis to be **reviewer-proof and realistic**, or **maximum ambitious from day one**? That changes glacier strategy completely.



Sources

focus on data downloading now

Perfect. Let's focus only on **data downloading architecture** and make it industrial-grade.

For your use case (**huge satellite data + clipping glacier AOI + retries + multiprocessing + seamless failures**), we should design this like a distributed ingestion system.

1. First: what exactly are we downloading?

From your methodology, these are the sources: Sentinel-1, Sentinel-2, MODIS/Landsat thermal, ERA5/MERRA-2, DEM, ALOS-2.

subsarfe\_melting\_line\_Methodolo...

But not all sources should be downloaded the same way.

We should split into **download categories**.

| Type          | Data         | Access Pattern                | Best Method            |
|---------------|--------------|-------------------------------|------------------------|
| Tile-based    | Sentinel-1/2 | spatial + date                | async workers          |
| API-based     | ERA5         | request → generate → download | queued retries         |
| Static        | DEM          | one-time                      | normal multiprocessing |
| Temporal bulk | MODIS        | many small files              | async                  |
| Hard source   | ALOS-2       | special portals               | isolated worker        |

## 2. Architecture decision

You said:

heavy multiprocessing + seamless data download + retry + RAM efficiency

The best stack is:

```
Async I/O
+
ProcessPoolExecutor
+
Disk-first pipeline
+
Persistent task queue
```



NOT Dask.

Final architecture:

```
Controller
|
SQLite Task DB
|
Async Download Manager (100+ tasks)
|
Retry Engine
|
Checksum Validator
|
Clipper Workers (multiprocessing)
|
Storage Writer
```



Why this architecture?

### Async for network

Downloading is I/O bound.

If you use multiprocessing only:

```
<> Python
Pool(100)
```



Run

you waste RAM because each process has Python overhead.

100 processes = RAM waste.

Instead:

```
100 async downloads
+
6-12 CPU workers
```



Much more efficient.

---

## 3. Use persistent queue (VERY IMPORTANT)

Never trust satellite APIs.

Things fail because of:

- rate limits
- timeout
- partial download
- broken connection
- server errors
- corrupt GeoTIFF

We need resumability.

### Use SQLite task database

Example table:

```
<> SQL   
  
downloads  
-----  
id  
source  
glacier  
date  
tile_id  
status  
retry_count  
filepath  
checksum  
last_error  
updated_at
```

Statuses:

```
  
PENDING  
RUNNING  
DOWNLOADED  
CLIPPED  
FAILED  
SKIPPED
```

If system crashes:

You restart script:

```
<> Bash   
  
python downloader.py
```

It resumes automatically.

No redownload.

This is a huge quality-of-life improvement.

---

## 4. Recommended concurrency

For your machine:

(16 GB RAM)

Recommended:

**Network workers**



50-150 async downloads



Very lightweight.

### CPU workers

For:

- reprojection
- clipping
- raster processing

Use:

</> Python



Run

```
max_workers = cpu_count() - 2
```

Likely:

6-10 workers



Never do:

40 multiprocessing workers



Rasterio/GDAL will kill RAM.

## 5. Download flow

Each acquisition should go through this exact pipeline:

```
Create task
  ↓
Check already exists
  ↓
Download
  ↓
Validate file
  ↓
Retry if broken
  ↓
Reproject CRS
  ↓
Clip glacier + buffer
  ↓
Save compressed output
  ↓
Mark complete
```



Never:

```
download everything
then process later
```



Disk explosion.

Instead:

### stream pipeline

download → process → delete temp



## 6. Retry policy (important)

Most people do bad retry systems.

We should use:

**exponential backoff + jitter**

Example:

```
Retry 1 → 3 sec  
Retry 2 → 8 sec  
Retry 3 → 20 sec  
Retry 4 → 60 sec  
Retry 5 → 180 sec  
Retry 6 → 10 min
```



Plus:

random jitter:

±20%



Prevents API throttling.

Permanent failure:

```
status = FAILED
```



Continue queue.

Never crash full pipeline.

## 7. RAM efficiency strategy

The biggest mistake:

```
<> Python  
raster = src.read()
```



Run

This loads huge rasters into memory.

Instead:

**windowed reading**

Use:

```
rasterio windows
```



or

```
rioxarray chunks
```



Clip only glacier extent.

Workflow:

```
open raster lazily
↓
crop bbox
↓
mask glacier polygon
↓
write compressed GeoTIFF
```



No full memory load.

## 8. Storage format decision

You must decide now.

### Option 1 — GeoTIFF only

Pros:

- simple
- GIS friendly

Cons:

- slower training pipeline

### Option 2 — Zarr (recommended)

Raw:

GeoTIFF



Processed:

Zarr



Why?

Jupyter training becomes insanely faster.

You can load:

```
<> Python
xarray.open_zarr()
```



Run

with lazy chunking.

Instead of:

300 GB into RAM



it loads only required chunks.

This matters massively for temporal windows.

## 9. Folder structure (important)

Do not keep random files.

Use:

```
glacier_project/  
  
config/  
glacier_shapes/  
  
data/  
  
    raw/  
        sentinel1/  
        sentinel2/  
        era5/  
        modis/  
        dem/  
        alos2/  
  
    temp/  
  
    clipped/  
        glacier_name/  
            year/  
                sensor/  
  
    processed/  
        aligned/  
  
metadata/  
    sqlite/  
    logs/  
    failed_tasks/  
  
notebooks/  
  
src/
```



Future you will thank yourself.

---

## 10. One important design decision before coding

We must decide:

### download strategy

#### Option A — Glacier-first

```
Khumbu  
    all sensors  
DrangDrung  
    all sensors
```



Pros:

- easy debugging
- smaller failures

Cons:

- slower globally

---

#### Option B — Sensor-first

```
download all Sentinel1  
download all Sentinel2
```



Pros:

- cleaner architecture
- reusable

Cons:

- harder debugging

For research work, I strongly recommend:

Glacier-first

Much easier to debug alignment issues.

Example:

```
Khumbu
2018-2024
↓
all modalities
↓
validate
↓
next glacier
```

Before next step, we need to lock this:

Are we doing glacier-first or sensor-first downloading?



Sources

i want all. give me a table from which source we are going to download. use copernicus and goole earth engine mainly alog with others source.

Good. If we are doing this seriously, we should lock the **complete data acquisition matrix** now.

We will prioritize **Copernicus + Google Earth Engine (GEE)** and use other sources where necessary.

Below is the **master download table** for the project.

Core Data Acquisition Table

| Data Category | Dataset                          | Source                                                                      | Platform       | Resolution | Temporal Frequency | Why Needed                              |
|---------------|----------------------------------|-----------------------------------------------------------------------------|----------------|------------|--------------------|-----------------------------------------|
| SAR (C-band)  | <a href="#">Sentinel-1 IW/EW</a> | <a href="#">Copernicus Data Space Ecosystem</a>                             | Copernicus     | 10-40 m    | 6-12 days          | dielectric sensing, wet snow, coherence |
| Optical       | <a href="#">Sentinel-2 MSI</a>   | <a href="#">Google Earth Engine</a> + <a href="#">Copernicus Data Space</a> | GEE/Copernicus | 10-60 m    | 5 days             | NDSI, albedo, grain size                |

|  | Data Category      | Dataset                                    | Source                                                 | Platform   | Resolution  | Temporal Frequency | Why Needed                          |
|--|--------------------|--------------------------------------------|--------------------------------------------------------|------------|-------------|--------------------|-------------------------------------|
|  |                    |                                            | <a href="#">Ecosystem</a>                              |            |             |                    |                                     |
|  | Thermal            | <a href="#">MODIS LST</a>                  | <a href="#">Google Earth Engine</a>                    | GEE        | 1 km        | Daily              | thermal lag, energy state           |
|  | Thermal (high-res) | <a href="#">Landsat 8/9 TIRS</a>           | <a href="#">Google Earth Engine</a>                    | GEE        | 100 m       | 16 days            | high-resolution surface temperature |
|  | Meteorology        | <a href="#">ERA5-Land</a>                  | <a href="#">Copernicus Climate Data Store</a>          | Copernicus | ~9 km       | Hourly             | energy balance forcing              |
|  | Meteorology backup | <a href="#">MERRA-2</a>                    | <a href="#">NASA GES DISC</a>                          | NASA       | ~50 km      | Hourly             | fallback forcing                    |
|  | DEM                | <a href="#">Copernicus DEM GLO-30</a>      | <a href="#">Copernicus Browser</a>                     | Copernicus | 30 m        | Static             | slope, aspect, radiation            |
|  | DEM backup         | <a href="#">TanDEM-X 90</a>                | <a href="#">German Aerospace Center (DLR) TanDEM-X</a> | DLR        | 90 m        | Static             | terrain refinement                  |
|  | Glacier Boundary   | <a href="#">Randolph Glacier Inventory</a> | <a href="#">RGI v7 Portal</a>                          | GLIMS      | polygon     | Static             | glacier clipping                    |
|  | Glacier Mask       | <a href="#">GLIMS</a>                      | <a href="#">GLIMS Glacier Database</a>                 | GLIMS      | polygon     | occasional         | cross-check boundaries              |
|  | Gravimetry         | <a href="#">GRACE-FO Mascons</a>           | <a href="#">NASA PO.DAAC</a>                           | NASA       | ~300 km     | Monthly            | mass-balance validation             |
|  | Elevation Change   | <a href="#">ICESat-2 ATL06</a>             | <a href="#">NASA Earthdata</a>                         | NASA       | track-based | repeat             | glacier thickness validation        |
|  | Snow Cover         | <a href="#">MOD10A1 Snow Product</a>       | <a href="#">Google Earth Engine</a>                    | GEE        | 500 m       | Daily              | snow dynamics                       |
|  | SAR (L-band)       | <a href="#">ALOS-2 PALSAR-2</a>            | <a href="#">Alaska Satellite Facility DAAC</a>         | ASF        | 25–100 m    | irregular          | deeper penetration (0–10 m)         |
|  |                    |                                            |                                                        |            |             |                    |                                     |

| Data Category          | Dataset                         | Source                                   | Platform | Resolution | Temporal Frequency | Why Needed        |
|------------------------|---------------------------------|------------------------------------------|----------|------------|--------------------|-------------------|
| Land Surface Radiation | <a href="#">CERES Radiation</a> | <a href="#">NASA CERES</a>               | NASA     | coarse     | daily              | energy validation |
| Weather Stations       | <a href="#">ICIMOD</a>          | <a href="#">ICIMOD Cryosphere Portal</a> | ICIMOD   | station    | hourly             | pseudo labels     |
| Weather Stations       | <a href="#">NOAA</a>            | <a href="#">NOAA Climate Data Online</a> | NOAA     | station    | hourly             | fallback AWS      |

## What we will actually automate

### Tier 1 — Fully automated (must-have)

These will be downloaded automatically:

```
Sentinel-1
Sentinel-2
MODIS LST
Landsat Thermal
ERA5-Land
Copernicus DEM
RGI glacier boundaries
GLIMS
```



These are enough to start training.

### Tier 2 — Semi-automated

```
ALOS-2
ICESat-2
GRACE-FO
ICIMOD AWS
```



These are more annoying APIs and inconsistent formats.

### Tier 3 — Validation only

```
CERES
MERRA-2
NOAA
```



Only if needed.

## Main platforms

We will mainly use:

### 1. Google Earth Engine

For:

Sentinel-2  
MODIS  
Landsat  
snow products  
quick clipping



Why:

- preprocessing already done
- cloud masking easier
- temporal filtering fast

## 2. Copernicus APIs

For:

Sentinel-1  
ERA5-Land  
Copernicus DEM



Why:

- original quality data
- no export limits
- better for SAR

## 3. ASF / NASA

For:

ALOS-2  
ICESat-2  
GRACE



## Proposed timeline for downloader implementation

### Phase 1

RGI polygons  
↓  
Sentinel-1  
↓  
Sentinel-2  
↓  
ERA5  
↓  
DEM  
↓  
MODIS



### Phase 2

Landsat thermal  
↓  
ICESat-2  
↓  
ALOS-2



### Phase 3



validation sources



This gives us a stable ingestion pipeline before touching difficult datasets.



↳ What we will actually automate

when ever i will run the full system i will give the date range







Good. We'll keep **both modes** and make them configurable at runtime.

## Failure handling modes

### 1. Strict Mode (Fail Fast)

Best for:

- debugging
- testing new source
- validating APIs
- checking clipping pipeline

Behavior:

```
One critical task fails
↓
retry exhausted
↓
STOP entire pipeline
```



Example:

```
❏ Bash

python run_pipeline.py \
  --start-date 2022-01-01 \
  --end-date 2022-12-31 \
  --mode strict
```



Output:

```
ERROR:
Sentinel-1 failed for Khumbu
Date: 2022-03-17

Pipeline stopped.
```



---

### 2. Resilient Mode (Production Mode)

Best for:

- long runs

- multi-year downloads
- unstable APIs
- overnight execution

Behavior:

```
task fails
↓
retry
↓
retry exhausted
↓
mark failed
↓
continue pipeline
```



At end:

```
download_report.json
```



Example report:

```
<> JSON

{
  "completed": 18392,
  "failed": 42,
  "skipped": 8123,
  "success_rate": "99.1%"
}
```



Run:

```
<> Bash

python run_pipeline.py \
  --start-date 2018-01-01 \
  --end-date 2024-12-31 \
  --mode resilient
```



---

## Final CLI Design

We should make the CLI flexible from day one.

### Minimal run

```
<> Bash

python run_pipeline.py \
  --start-date 2018-01-01 \
  --end-date 2024-12-31
```



Uses default glaciers + default sources.

---

### Full custom run

```
<> Bash

python run_pipeline.py \
  --start-date 2018-01-01 \
```



```
--end-date 2024-12-31 \  
--glaciers khumbu drang_drung helheim \  
--sources sentinel1 sentinel2 era5 modis dem \  
--mode resilient \  
--max-downloads 100 \  
--cpu-workers 8
```

---

## Debug mode

Single glacier + strict

```
</> Bash   
  
python run_pipeline.py \  
  --glaciers khumbu \  
  --start-date 2023-01-01 \  
  --end-date 2023-03-01 \  
  --mode strict
```

---

## Internal config hierarchy

Priority:

```
CLI args   
>  
config.yaml  
>  
default settings
```

Meaning:

You can permanently save defaults in config.

But override anytime from terminal.

Example:

```
</> YAML   
  
default_mode: resilient  
max_downloads: 100  
cpu_workers: 8  
  
sources:  
  sentinel1: true  
  sentinel2: true  
  era5: true  
  modis: true
```

Then:

```
</> Bash   
  
--mode strict
```

overrides config temporarily.

---

One more decision before architecture freeze:

Should we support **checkpoint restart**?

Example:

Pipeline crashes at:

78%



You rerun:

```
</> Bash
python run_pipeline.py
```



and it resumes from:

78%



instead of starting from zero.

I strongly recommend **yes** for multi-year satellite downloads.



do one thing make a small qt5 ui for as control and show the progress details.

Yes — that is the right move for a long-running downloader.

For a project like this, a **small PyQt5 control panel** is much better than only CLI because downloads may run for **hours/days**, and you'll want:

- monitor progress
- pause/resume
- see failures
- restart failed jobs
- live logs
- glacier/source status
- ETA
- CPU/RAM/network usage

But we should keep it **thin** — UI as a controller only. The heavy downloader stays backend-driven.

## Recommended architecture

```
PyQt5 UI
|
Controller Layer
|
Download Engine
(async + multiprocessing)
|
SQLite State DB
```



Meaning:

The UI never directly downloads anything.



UI only:

```
start
pause
resume
cancel
retry failed
monitor progress
```

This makes it stable.

If UI crashes:

```
downloads continue running
```

Huge advantage.

## Proposed UI Layout

### Left Panel — Run Configuration

Controls:

```
Start Date
End Date

Glacier Selection
☒ Khumbu
☒ Drang-Drung
☒ Helheim

Sources
☒ Sentinel-1
☒ Sentinel-2
☒ ERA5
☒ MODIS
☒ DEM
☐ ALOS-2

Mode:
(•) Resilient
( ) Strict

Parallelism
Downloads: [100]
CPU Workers: [8]
Retries: [8]
```

Buttons:

```
[Start]
[Pause]
[Resume]
[Stop]
[Retry Failed]
```

### Center Panel — Live Progress

Global progress bar:



```
Downloaded: 18,430
Skipped: 2,920
Failed: 31
Running: 44
Pending: 9,211
```



Estimated Time:  
13h 42m



Download Speed:  
182 MB/s



```
CPU Usage
RAM Usage
Disk Usage
Network Speed
Queue Size
```



```
[21:04:11] Sentinel-1 downloaded
Khumbu | 2021-03-04

[21:04:17] ERA5 retry attempt 2

[21:04:31] Glacier clip complete
```



ALL  
INFO  
WARNING  
ERROR



Expandable table:

| Glacier | Source | Progress | Failed | Status   |
|---------|--------|----------|--------|----------|
| Khumbu  | S1     | 72%      | 1      | Running  |
| Khumbu  | S2     | 84%      | 0      | Running  |
| Helheim | ERA5   | 21%      | 4      | Retrying |

This becomes very useful.

## Important design choice

We should make the UI:

### Option A — Minimal functional UI

Simple and fast.

~1 day build.

Looks basic but robust.

### Option B — Professional dashboard

Modern dark theme.

- cards
- charts
- live plots
- RAM graph
- throughput graph
- cleaner UX

~3–5x more effort.

Since this is your thesis/research infra and you'll use it repeatedly, I recommend a **clean professional UI but lightweight**, not overdesigned.

Also, I would suggest **PySide6 instead of PyQt5**.

Why:

```
better licensing
modern Qt
better future support
same API mostly
```



Much safer for a serious project.

One more important decision:

Should the downloader **continue in background even if the UI closes**, or should closing UI stop everything?

I strongly recommend:

```
UI closes
↓
backend continues
↓
```



```
reopen UI
↓
reattach to running job
```

This is much more robust for multi-day downloads.



from inputGeoJson folder, it will read all the geojson from that folder. number of files is number of glacier and file name will be glacier name. make a config file, to manually control the cpu and ram load. if possible use the gpu also for processing. create a config file for the percentage that the system will use.

Perfect. Now the architecture is becoming solid.

We'll make this a **config-driven autonomous glacier ingestion engine**.

No hardcoded glacier names.

## Final glacier input design

Folder:

```
inputGeoJson/
```



Example:

```
inputGeoJson/
```



```
khumbu.geojson
drang_drung.geojson
helheim.geojson
siachen.geojson
```

System behavior:

```
read all *.geojson
↓
filename = glacier name
↓
AOI extracted
↓
buffer applied
↓
task generation
```



Example:

```
drang_drung.geojson
```



becomes:

```
glacier_name = drang_drung
```



No need to manually add glaciers.

Add file → glacier automatically included.

Remove file → excluded.

Much cleaner.

---

## GeoJSON requirements

Every glacier file:

```
«> JSON
{
  "type": "FeatureCollection",
  "features": [...]
}
```

System will:

### validate geometry

Check:

```
valid polygon
non-empty geometry
valid CRS
repair invalid geometry
```

Then convert to:

```
EPSG:4326
```

internally.

---

## Resource control system (VERY IMPORTANT)

You asked:

manually control cpu and ram load

This is absolutely the right approach.

We'll make a **resource-aware scheduler**.

Instead of:

```
max_workers=8
```

we do:

```
dynamic adaptive concurrency
```

based on config.

---

## Config-driven resource limits

Example:

```
«> YAML
```

```
system:

cpu:
  enabled: true
  max_usage_percent: 70
  reserve_cores: 2

ram:
  enabled: true
  max_usage_percent: 65
  emergency_stop_percent: 90

gpu:
  enabled: true
  max_usage_percent: 50
  use_for:
    - raster_processing
    - reprojection
    - clipping

disk:
  min_free_space_gb: 100

network:
  max_download_mbps: 0
```

Meaning:

## CPU

If CPU exceeds:

70%



scheduler slows workers.

Example:

```
100 downloads
↓
cpu overload
↓
reduce clipping workers
```



No freezing system.

---

## RAM

If RAM exceeds:

65%



system throttles.

Example:

```
pause raster processing
keep downloading
```



Emergency:

90%



Stop heavy jobs immediately.

Prevents crashes.

---

## GPU support (important reality check)

You asked:

if possible use gpu also for processing

**YES — selectively.**

But GPU is NOT useful for everything.

### Good GPU candidates

#### Raster reprojection

Can use:

```
cuSpatial  
CuPy  
RAPIDS
```



Potential acceleration.

---

#### Raster clipping

Possible GPU acceleration.

Especially large Sentinel rasters.

---

#### Resampling

GPU helps.

---

### BAD GPU candidates

#### Downloading

No benefit.

---

#### API calls

No benefit.

---

#### GeoJSON parsing

No benefit.

---

### Recommended GPU design

```
</> YAML  
  
gpu:  
  
  enabled: true
```



```
acceleration:

  raster_clip: true
  reprojection: true
  resampling: true
  cloud_masking: true

memory_limit_percent: 60
```

If GPU unavailable:

Automatic fallback:

CPU mode



No crash.

---

## Adaptive Scheduler Logic

Instead of fixed workers:

System behaves like this:

Example:

### Situation 1

CPU: 32%  
RAM: 40%



Scheduler:

downloads = 120  
cpu workers = 8



---

### Situation 2

CPU: 80%  
RAM: 72%



Scheduler:

downloads = 60  
cpu workers = 3



Automatically.

---

### Situation 3

RAM: 91%



Emergency mode:

pause raster processing  
wait for memory cleanup





## Full config structure

Something like:

```
</> YAML   
  
project:  
  output_dir: "./data"  
  geojson_dir: "./inputGeoJson"  
  
date_range:  
  start: "2018-01-01"  
  end: "2024-12-31"  
  
download:  
  
  mode: resilient  
  
  retries: 8  
  
  async_downloads: 100  
  
  checkpoint_restart: true  
  
sources:  
  
  sentinel1: true  
  sentinel2: true  
  modis_lst: true  
  landsat_thermal: true  
  era5: true  
  dem: true  
  alos2: false  
  grace: false  
  icesat2: false  
  
processing:  
  
  glacier_buffer_km: 5  
  
  compression: "LZW"  
  
  output_format: "zarr"  
  
resource_control:  
  
  cpu:  
    enabled: true  
    max_usage_percent: 70  
    reserve_cores: 2  
  
  ram:  
    enabled: true  
    max_usage_percent: 65  
    emergency_stop_percent: 90  
  
  gpu:  
    enabled: true  
    max_usage_percent: 50  
  
ui:  
  
  refresh_rate_ms: 1000  
  dark_mode: true  
  show_live_logs: true
```

This makes the whole system **fully configurable without changing code**.

One recommendation:

Instead of percentage-only CPU, we should also allow:

```
reserve_cores
```

So your laptop/desktop never becomes unusable.

Example:

```
</> YAML
reserve_cores: 2
```

Meaning:

System always leaves:

```
2 CPU cores free
```

for normal work/browser/Jupyter.

That will make your machine much more usable during multi-day downloads.



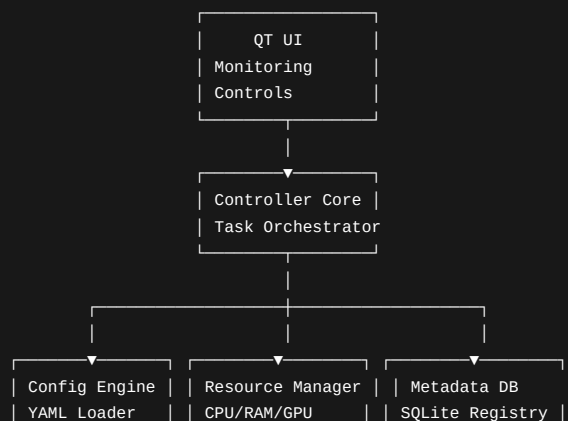
create now make the full system design. ignore the glacier names now. also in filename of geojson it will be written that from which source the data will be downloaded example  
drang\_drung\_copurnicus\_sentinel2.geojson create a config file which will have that from the satellite which band will be downloaded.

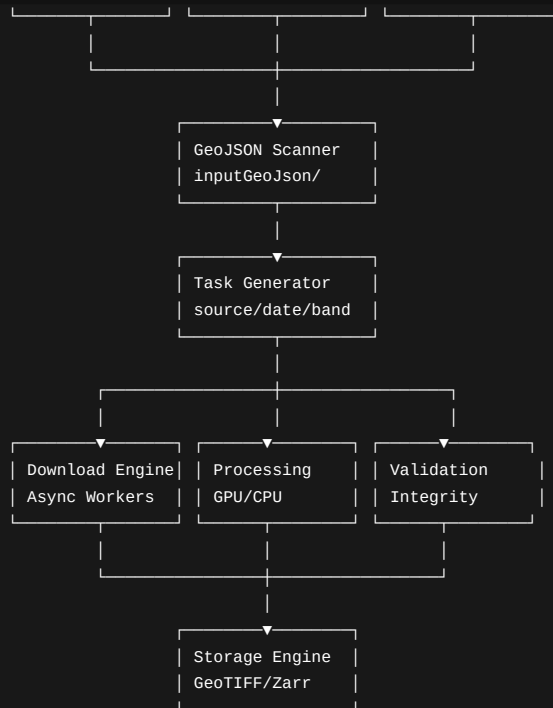
Excellent. Now we can design the **full production-grade system architecture**.

We'll ignore glacier names and make it **fully autonomous + config driven + geojson driven + satellite-band configurable**.

This will behave like a mini remote sensing platform.

## 1. High-Level System Architecture





## 2. GeoJSON naming system

You proposed:

drang\_drung\_copurnicus\_sentinel2.geojson



We'll formalize it.

### Final naming convention

<glacier\_name>\_<platform>\_<dataset>.geojson



Examples:

khumbu\_copernicus\_sentinel1.geojson  
khumbu\_copernicus\_sentinel2.geojson  
khumbu\_gee\_modis\_lst.geojson  
khumbu\_cds\_era5.geojson  
khumbu\_asf\_alos2.geojson



Meaning:

#### filename determines:

1. glacier name
2. platform
3. dataset
4. automatic downloader

No hardcoding.

### Parsing logic

Example:

```
drang_drung_copernicus_sentinel2.geojson
```



Parsed as:

```
<> JSON
```



```
{
  "glacier_name": "drang_drung",
  "platform": "copernicus",
  "dataset": "sentinel2"
}
```

Then system auto routes:

```
sentinel2
```

```
↓
```

```
Copernicus Downloader
```



### 3. inputGeojson folder

Structure:

```
inputGeoJson/
```



```
glacierA_copernicus_sentinel1.geojson
glacierA_copernicus_sentinel2.geojson
glacierA_cds_era5.geojson
```

```
glacierB_gee_modis_lst.geojson
glacierB_copernicus_dem.geojson
```

System behavior:

```
scan folder
```

```
↓
```

```
read filenames
```

```
↓
```

```
generate task graph
```

```
↓
```

```
validate geometry
```

```
↓
```

```
build jobs
```



### 4. Config-driven satellite band system

You requested:

```
config file which will have that from satellite which band will be downloaded
```

This is the right approach.

#### Final config design

```
config.yaml
```

```
<> YAML
```



```
project:
```

```
  input_geojson_folder: "./inputGeoJson"
```

```
    output_dir: "./data"

date_range:

    start_date: "2018-01-01"
    end_date: "2024-12-31"

download:

    mode: resilient
    retries: 8
    checkpoint_restart: true

parallelism:

    async_downloads: 100
    cpu_workers: auto

resource_control:

    cpu:
        enabled: true
        max_usage_percent: 70
        reserve_cores: 2

    ram:
        enabled: true
        max_usage_percent: 65
        emergency_stop_percent: 90

    gpu:
        enabled: true
        max_usage_percent: 60
        auto_fallback_cpu: true

sources:

    sentinel1:
        enabled: true

        product_type:
            - GRD

        polarization:
            - VV
            - VH

        orbit:
            - ASCENDING
            - DESCENDING

        resolution_m: 10

        coherence:
            enabled: true
            temporal_days:
                - 12
                - 24

    sentinel2:
        enabled: true

        cloud_cover_max: 25

        atmospheric_correction: true

        bands:
            - B2
            - B3
```

```
- B4
- B8
- B8A
- B11
- B12

indices:
  - ndsi
  - ndwi
  - albedo

resolution_m: 10

modis_lst:
  enabled: true

bands:
  - LST_Day_1km
  - LST_Night_1km

landsat_thermal:
  enabled: true

satellites:
  - Landsat8
  - Landsat9

bands:
  - ST_B10

era5:
  enabled: true

variables:
  - 2m_temperature
  - snowfall
  - snow_depth
  - total_precipitation
  - skin_temperature
  - surface_pressure
  - shortwave_radiation
  - longwave_radiation
  - sensible_heat_flux
  - latent_heat_flux
  - wind_u
  - wind_v

temporal_resolution: hourly

dem:
  enabled: true

  source: copernicus_dem_glo30

derived_products:
  - slope
  - aspect
  - curvature
  - hillshade
  - sky_view_factor

processing:

  glacier_buffer_km: 5

  reprojection_crs: EPSG:4326

  compression: LZW
```

```
output_format:

  raw: geotiff
  processed: zarr

storage:

  delete_temp_files: true

  skip_existing_files: true

ui:

  refresh_rate_ms: 1000
  dark_mode: true
```

---

## 5. Band-aware downloader logic

Example:

### Sentinel-2

Config says:

```
</> YAML

bands:
- B2
- B3
- B4
- B8
```



System only downloads:

```
Blue
Green
Red
NIR
```



Skipping unnecessary bands.

Huge storage saving.

---

### Sentinel-1

Config:

```
</> YAML

polarization:
- VV
- VH
```



Only VV/VH downloaded.

No HH/HV waste.

---

### ERA5

Only requested variables downloaded.

Instead of:

entire weather cube



Huge bandwidth saving.

## 6. Folder structure

Final structure:

```
project_root/

|— config/
|   config.yaml

|— inputGeoJson/

|— data/

|   raw/
|       sentinel1/
|       sentinel2/
|       era5/
|       modis/
|       dem/

|   clipped/

|   processed/
|       zarr/

|— metadata/

|   sqlite/
|       registry.db

|   logs/

|   reports/

|— ui/

|— src/

|— notebooks/
```



## 7. End-to-end execution flow

When you click:

START



UI sends:

```
date range
config
resource settings
```



System:



```
1. scan geojson folder
2. parse filename
3. validate polygon
4. query source
5. create task graph
6. download
7. verify integrity
8. clip glacier AOI
9. process bands
10. save output
11. update SQLite
12. update UI
```



Crash?

Resume:

from checkpoint



automatically.

This is now close to an **industrial-grade remote sensing ingestion platform**.



↳ This is now close to an **industrial-grade remote sensing ingestion platform**.

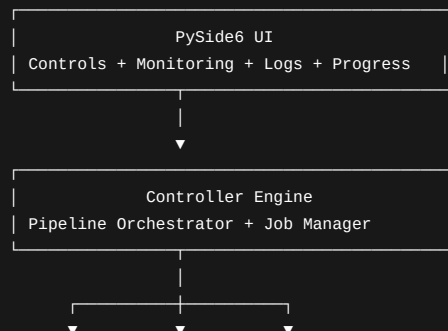
give me the system archtechure with the technology we will use

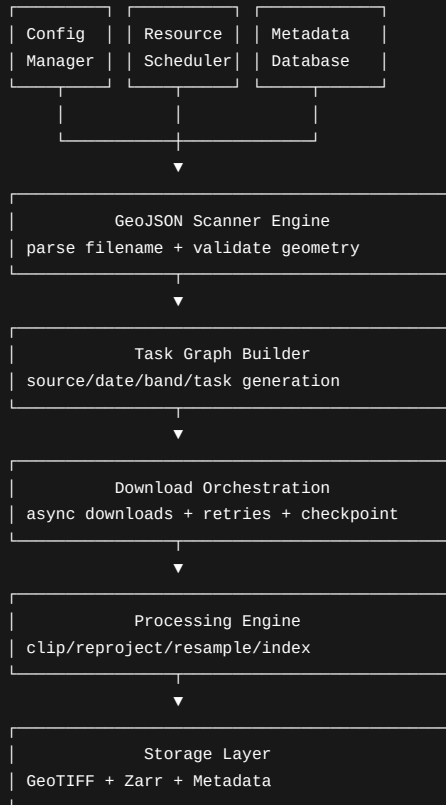
Perfect. Below is the **complete industrial-grade system architecture** with the **technology stack** we will use for each component.

This is optimized for:

- huge satellite downloads
- multiprocessing
- RAM efficiency
- resumability
- GPU acceleration (where useful)
- PyQt control panel
- multi-glacier scaling
- future ML integration with Jupyter

## 1. High-Level System Architecture





## 2. Technology Stack

### A. UI Layer

#### Technology

PySide6

Why:

- modern Qt
- professional desktop UI
- better licensing than PyQt5
- stable for long-running apps
- multithreading support

#### Responsibilities

Start pipeline  
Pause  
Resume  
Stop  
Retry failed  
Live logs  
Progress bars  
CPU/RAM/GPU monitoring  
ETA estimation



#### Additional libraries

| Purpose      | Technology          |
|--------------|---------------------|
| Modern theme | qdarktheme          |
| Charts       | pyqtgraph           |
| Tables       | Qt Model View       |
| Logging UI   | QTextEdit streaming |

## B. Configuration System

### Technology

PyYAML

Config:

```
config.yaml
```



Why:

- human editable
- easy experimentation
- no code changes required

Controls:

```
bands
resources
date_range
parallelism
GPU
retry_policy
storage
```



## C. Orchestrator / Backend Core

### Technology

Pure Python

Architecture:

```
service-based modular architecture
```



Modules:

```
controller/
scheduler/
task_manager/
download_engine/
processing_engine/
storage_engine/
```



Why:

Maximum control.

No framework overhead.

## D. Download Engine

### Technology Stack

| Purpose          | Technology     |
|------------------|----------------|
| Async networking | Python asyncio |
| HTTP requests    | aiohttp        |
| Retry engine     | tenacity       |
| Queue management | asyncio.Queue  |

Why:

Downloading satellite data is:

network-bound



not CPU-bound.

### Design

100-300 async downloads  
+  
limited CPU processing workers



This is much more RAM efficient.

## E. Multiprocessing Layer

### Technology

Python:

```
<> Python
concurrent.futures.ProcessPoolExecutor
```



Run

NOT normal multiprocessing pool.

Why:

Better:

crash handling  
future management  
worker monitoring



Used for:

clipping  
reprojection  
resampling  
compression



band extraction  
terrain generation

## F. GPU Acceleration

### Technology Stack

| Task                 | Technology       |
|----------------------|------------------|
| GPU arrays           | CuPy             |
| Spatial acceleration | RAPIDS cuSpatial |
| Raster compute       | Numba CUDA       |
| ML preprocessing     | PyTorch CUDA     |

### Used for

large raster clipping  
resampling  
DEM derivatives  
band math  
terrain products



### Automatic fallback

If GPU unavailable:

CPU processing



No failure.

## G. Geospatial Engine

### Core Technologies

| Purpose                   | Technology    |
|---------------------------|---------------|
| Raster handling           | rasterio      |
| Vector handling           | geopandas     |
| Geometry ops              | shapely       |
| CRS handling              | pyproj        |
| Spatial clipping          | rioxarray     |
| Large raster lazy loading | xarray        |
| Metadata                  | pystac-client |

Why this stack?

Industrial-grade geospatial pipeline.

## H. Heavy Raster Backend

### Technology

[GDAL Official Website](#)

Everything underneath runs on:

GDAL



Critical for:

GeoTIFF  
reprojection  
warping  
compression  
resampling



We'll use:

LZW compression  
COG optimization



for storage efficiency.

## I. Metadata & Checkpointing

### Technology

[SQLite](#)

Database:

registry.db



Stores:

| Table      | Purpose        |
|------------|----------------|
| downloads  | task tracking  |
| failures   | retry tracking |
| glaciers   | metadata       |
| processing | clip status    |
| storage    | file registry  |

Why SQLite?

- zero setup
- fast
- reliable
- resumable

## J. Resource Monitoring

### Technology

| Purpose | Technology      |
|---------|-----------------|
| CPU/RAM | psutil          |
| GPU     | pynvml          |
| Disk    | shutil + psutil |
| Network | psutil          |

Scheduler logic:

```

CPU high
↓
reduce workers

RAM high
↓
pause raster processing

GPU high
↓
fallback CPU

```



## K. Satellite APIs

### 1. Copernicus

Main source.

For:

```

Sentinel-1
Sentinel-2
DEM
ERA5

```



Tech:

```

Copernicus STAC API

```



Authentication:

```

OAuth2

```



### 2. Google Earth Engine

For:

```

MODIS
Landsat
Snow products

```



Technology:

[Google Earth Engine API](#)

Python SDK.

### 3. NASA APIs

For:

ICESat-2  
GRACE  
MERRA-2



Tech:

EarthData API



### 4. ASF API

For:

ALOS-2



Technology:

ASF Search API



## L. Storage Architecture

### Raw Storage

Format:

Cloud Optimized GeoTIFF (COG)



Why:

- GIS compatible
- chunked
- efficient

### Processed Storage

Format:

Zarr

Why:

Jupyter training becomes fast.

Supports:

Python



```
xarray.open_zarr()
```

Run

without RAM explosion.



## M. Logging System

### Technology

Python:

logging



Output:

console  
UI  
JSON logs  
error logs



Log levels:

INFO  
WARNING  
ERROR  
CRITICAL



## 3. Folder Architecture

```
project/

|— config/
|   config.yaml

|— inputGeoJson/

|— data/

|   raw/
|       sentinel1/
|       sentinel2/
|       era5/
|       modis/
|       dem/

|   clipped/

|   processed/
|       zarr/

|— metadata/

|   sqlite/
|       registry.db

|   logs/
|   reports/

|— src/

|   controller/
|   scheduler/
|   downloaders/
|   processors/
|   storage/
|   monitoring/
|   ui/
```



## 4. Execution Pipeline

When you click **START**:

```
1. Read config.yaml
2. Scan inputGeoJson
3. Parse filenames
4. Validate polygons
5. Generate task graph
6. Query APIs
7. Download
8. Verify checksum
9. Clip glacier polygon
10. Process bands
11. Save GeoTIFF
12. Convert to Zarr
13. Update SQLite
14. Update UI
15. Generate report
```



Crash?

```
restart
↓
resume checkpoint
```



No re-download.

This architecture is strong enough to scale from **one glacier** → **hundreds of glaciers** without redesign.



also add qt5 design description. make it colorfull and butifull. use combination of mat black, mat purpel and gold.

Perfect. Since this will run for long hours and you'll use it repeatedly, the UI should feel like a **professional remote sensing control center**, not a generic developer app.

We'll design a **beautiful research-grade QT5/PySide6 desktop dashboard** using:

### Theme palette

|               |                             |
|---------------|-----------------------------|
| Primary Base  | → Matte Black               |
| Secondary     | → Matte Purple              |
| Accent        | → Metallic Gold             |
| Text          | → Soft White                |
| Status Colors | → Emerald / Amber / Crimson |



Design goal:

```
NASA Mission Control
+
```



Not flashy, but premium.

# 1. Color Palette

## Primary Colors

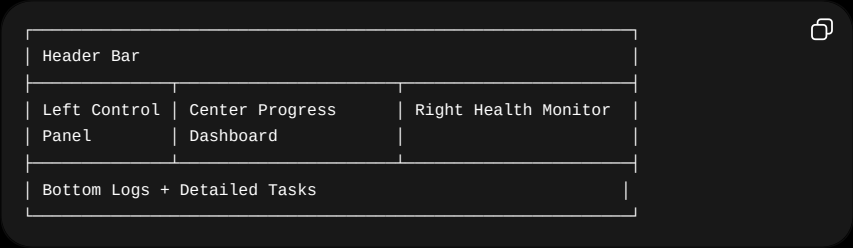
| Element      | Color          | Hex     |
|--------------|----------------|---------|
| Matte Black  | background     | #121212 |
| Deep Black   | cards/panels   | #1A1A1A |
| Matte Purple | secondary      | #5E3A87 |
| Deep Purple  | hover          | #4B2E6E |
| Royal Gold   | accent         | #D4AF37 |
| Soft Gold    | hover accent   | #E6C15A |
| White Smoke  | text           | #ECECEC |
| Dim Gray     | secondary text | #A8A8A8 |

Status colors:

| State   | Color   |
|---------|---------|
| Success | #00C896 |
| Warning | #F4B400 |
| Error   | #FF5C5C |
| Running | #3DA5FF |

# 2. Full QT5 Window Layout

We'll use a 3-column professional dashboard.



Window size:

1600 × 900

Resizable

Dark Mode Only

Rounded modern UI.

### 3. Header Bar (Top)

Purpose:

Mission-control feel.

Layout:



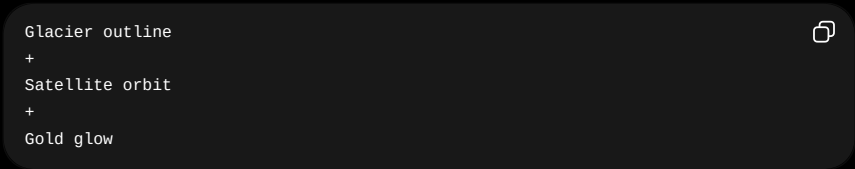
Style:

#### Background

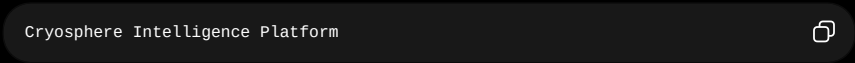
Matte black gradient.

#### Left Logo

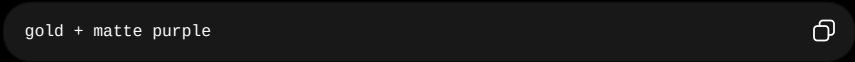
Custom icon:



Text:

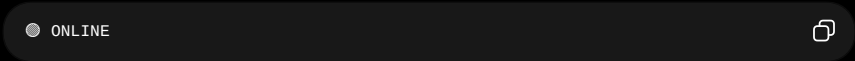


Color:



Right:

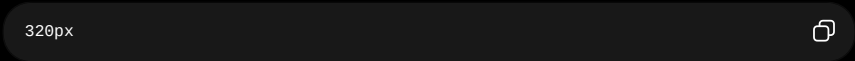
System status:



animated pulse.

### 4. Left Control Panel

Width:



Purpose:

Pipeline control.

#### Section 1 — Date Range

Card:

Date Range

Start [Calendar]

End [Calendar]



Design:

- rounded corners
- purple border glow
- gold icons

## Section 2 — Resource Control

Sliders:

CPU Usage

[====70%==]

RAM Usage

[===65%===]

GPU Usage

[====50%==]

Downloads

[==100=====]



Style:

Gold progress sliders.

Hover:

purple glow.

## Section 3 — Download Sources

Beautiful toggle cards.

Example:

☒ Sentinel-1

☒ Sentinel-2

☒ ERA5

☒ MODIS

☐ ALOS-2



Instead of ugly checkboxes:

Modern pill switches.

Purple active.

Gold selected.

## Section 4 — Execution Mode

Beautiful segmented buttons:

Resilient

Strict



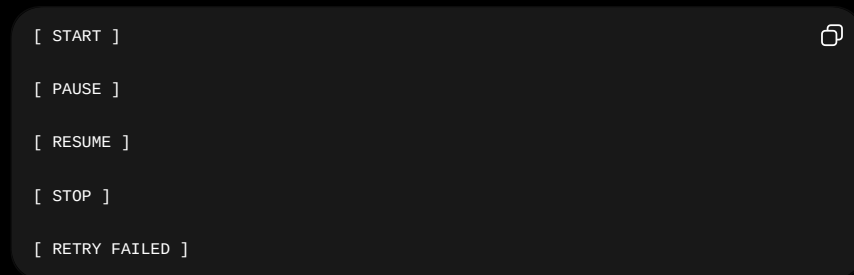
Selected:

gold highlight.

---

## Section 5 — Main Buttons

Large premium buttons.



Design:

### Start

Gold gradient.

### Pause

Purple.

### Stop

Deep crimson.

Hover:

soft glow.

Rounded:



---

## 5. Center Progress Dashboard

This is the main hero section.

Large elegant cards.

### Overall Progress

Massive circular progress ring:



Animated.

Gold outer ring.

Purple inner glow.

Below:



Beautiful graph:



Purple line.

Gold points.

Large typography:



Cards:



Gold progress fill.

## Glassmorphism cards.

Circular radial gauge.



Purple fill.

## RAM

RAM  
58%



Gold fill.

## GPU

GPU  
22%  
CUDA ACTIVE



Animated.

## Disk

Disk Remaining  
3.4 TB



## Queue Health

Pending Tasks  
4,112

Retries  
28

Failed  
7



## 7. Bottom Section

Two tabs.

### Tab 1 — Live Logs

Professional terminal.

Example:

```
[21:44:13] Sentinel-1 downloaded  
Khumbu | 2024-02-13
```

```
[21:44:17] ERA5 retry attempt 2
```

```
[21:44:29] Clip complete
```



Color coding:

### Info

Soft white.

### Warning



Amber.

Error

Red.

Success

Emerald green.

Search bar:

[ Search logs ]

Filter:

ALL  
INFO  
WARNING  
ERROR

Tab 2 — Detailed Task Table

Professional table:

| Glacier  | Source | Date    | Progress | Status  |
|----------|--------|---------|----------|---------|
| GlacierA | S1     | 2024-01 | 78%      | Running |
| GlacierB | S2     | 2023-04 | 100%     | Done    |

Status badges:

RUNNING  
FAILED  
RETRYING  
DONE

Beautiful chips.

8. Visual Effects

Subtle only.

Hover glow

Purple.

Active glow

Gold.

Smooth animations

Qt animations:

fade  
slide  
pulse

Nothing excessive.

## 9. Typography

Font:

Inter  
or  
SF Pro



Header:

Gold.

Body:

Soft white.

Numbers:

Bold futuristic style.

## 10. Design Language

Style inspiration:

Mix of:

- Bloomberg Terminal (clean data density)
- Unreal Engine launcher (premium dark UI)
- Cyberpunk HUD (subtle)
- NASA control software

Avoid:

cheap gamer RGB look



Goal:

Professional  
Research-grade  
Premium  
Long-hour comfortable



This will look like a serious scientific platform rather than a hobby tool.



Is this conversation helpful so far?